

Arquitetura Algorítmica e Configuração da Meta-Heurística — SCCVRPSD com Jsprit

Nota sobre fontes: Cada seção indica a origem da informação entre [CÓDIGO] (verificável diretamente no código-fonte deste projeto), [JSPRIT-SRC] (código-fonte do Jsprit no GitHub) ou [JSPRIT-DOC] (documentação oficial do Jsprit). Referências completas na Seção 6.

1. Estratégia Geral da Meta-Heurística

O solver utilizado é o **Jsprit v1.9.0-beta.3** (GraphHopper) [CÓDIGO: `pom.xml:32-33`], instanciado através da classe `Jsprit.Builder.newInstance(problem).buildAlgorithm()` [CÓDIGO: `VrpService.java:404,441`].

O Jsprit implementa internamente uma meta-heurística baseada no paradigma **Ruin and Recreate** (Schrimpf et al., 2000), que é uma **Large Neighbourhood Search (LNS)** combinando elementos de *Simulated Annealing* e *Threshold Accepting*. Conforme a documentação oficial do Jsprit:

"The meta-heuristic applied to solve vehicle routing problems with jsprit was developed by Schrimpf et al. (2000) who formulated the ruin-and-recreate principle. It is a large neighborhood search that combines elements of simulated annealing and threshold-accepting algorithms." — [JSPRIT-DOC: `docs/Meta-Heuristic.md`]

A implementação estende o paradigma original com estratégias adicionais inspiradas em Pisinger e Ropke (2007) [JSPRIT-DOC: `docs/Meta-Heuristic.md`].

O critério de aceitação de soluções vizinhas é baseado em **threshold acceptance** (classe `SchrimpfAcceptance`), onde soluções piores são aceitas se estiverem dentro de um limiar dinâmico que decai **exponencialmente** ao longo das iterações [JSPRIT-SRC: `SchrimpfAcceptance.java`]:

$$\text{threshold}(i) = \text{initialThreshold} \times \exp(-\ln(2) \times (i / \text{maxIterations}) / \alpha)$$

Onde $\text{initialThreshold} = \text{custoMelhorSolução} \times \theta_0$ e $\alpha = \text{THRESHOLD_ALPHA}$.

Adicionalmente, o projeto implementa uma **estratégia multi-start** externa ao Jsprit: múltiplas execuções independentes do algoritmo são disparadas com sementes aleatórias distintas, e a melhor solução global é retida [CÓDIGO: `VrpService.java:399-425, 430-461`].

2. Construção da Solução Inicial

A heurística de construção padrão do Jsprit é a **Regret Insertion** (parâmetro interno `CONSTRUCTION = REGRET_INSERTION`) [JSPRIT-SRC: `Jsprit.java`, propriedade padrão `CONSTRUCTION`]. Nesta abordagem:

1. Todos os jobs (Delivery e Pickup) iniciam como não atribuídos.
2. Para cada job não atribuído, calcula-se o *regret value*: a diferença entre o custo da segunda melhor inserção e o custo da melhor inserção. O parâmetro padrão é Regret-2 (`REGRET_K = 2`) com estratégia

- de soma (`REGRET_K_STRATEGY = "sum"`) [JSPRIT-SRC: `Jsprit.java`].
3. O job com **maior regret** (ou seja, que mais "perderia" se não fosse inserido na sua melhor posição agora) é inserido primeiro.
 4. O processo repete até que todos os jobs sejam inseridos ou nenhuma inserção viável exista.

O parâmetro `FAST_REGRET` está configurado como `"false"` [CÓDIGO: `VrpService.java:407,444`]. Isto **não desabilita** a Regret Insertion — apenas indica que a versão padrão (não otimizada) é utilizada, ao invés da variante `RegretInsertionFast` que aplica filtragem espacial para acelerar a avaliação [JSPRIT-SRC: `Jsprit.java`; JSPRIT-DOC: GraphHopper Forum, tópico `"jsprit-parameter-fast-regret"`].

3. Operadores de Vizinhança (Ruin & Recreate)

3.1 Estratégias de Destruição (Ruin)

O `Jsprit.Builder` configura automaticamente os seguintes operadores de destruição [JSPRIT-SRC: `Jsprit.java`, método `createDefaultProperties()`]:

Operador	Classe Jsprit	Peso padrão (c/ Regret)	Peso padrão (c/ Best)	Descrição
Random Ruin	<code>RuinRandom</code>	0.5	0.5	Remove jobs aleatoriamente
Radial Ruin	<code>RuinRadialDynamic</code>	0.5	0.0	Remove jobs geograficamente próximos a um job-semente
Worst Ruin	<code>RuinWorst</code>	1.0	0.0	Remove jobs com maior custo marginal na rota
Cluster Ruin (DBSCAN)	<code>RuinClusters</code>	1.0	0.0	Remove clusters de jobs usando DBSCAN
Cluster Ruin (Kruskal)	<code>RuinKruskalClusters</code>	0.0 (desab.)	0.0	Remove clusters via árvore geradora mínima
String Ruin	<code>RuinString</code>	0.0 (desab.)	0.0	Remove sequências contíguas de jobs
Time-Related Ruin	<code>RuinTimeRelated</code>	0.0 (desab.)	0.0	Remove jobs com janelas temporais similares

Os operadores com peso > 0 são selecionados em cada iteração. Os operadores com peso 0 estão desabilitados por padrão.

Frações de remoção [JSPRIT-SRC: `Jsprit.java`]:

- **Radial, Worst e Cluster Ruin:** mínimo = `max(3, 5% dos jobs)` (teto 20); máximo = `max(5, 30% dos jobs)` (teto 50).
- **Random Ruin:** fração fixa de **50% dos jobs** (min e max iguais), com teto de 70 jobs.

```
// Radial, Worst, Cluster
int minShare = (int) Math.min(20, Math.max(3, vrp.getJobs().size() * 0.05));
int maxShare = (int) Math.min(50, Math.max(5, vrp.getJobs().size() * 0.3));

// Random
int minShare_ = (int) Math.min(70, Math.max(5, vrp.getJobs().size() * 0.5));
int maxShare_ = (int) Math.min(70, Math.max(5, vrp.getJobs().size() * 0.5));
```

3.2 Estratégias de Reconstrução (Recreate / Insertion)

Após a destruição, os jobs removidos são reinseridos. O Jsprit acopla cada operador de ruin a uma estratégia de inserção [JSPRIT-SRC: Jsprit.java]:

Estratégia	Classe Jsprit	Uso
Regret Insertion	<code>RegretInsertion</code> / <code>RegretInsertionConcurrent</code>	Acoplada a Random Ruin, Radial Ruin, Worst Ruin e Cluster Ruin (com pesos > 0)
Best Insertion	<code>BestInsertion</code> / <code>BestInsertionConcurrent</code>	Acoplada a Random Ruin (segunda instância, com peso 0.5)
Cheapest Insertion	<code>CheapestInsertion</code> / <code>CheapestInsertionConcurrent</code>	Disponível, mas com peso 0.0 por padrão (desabilitada)

A Regret Insertion prioriza jobs com poucas boas opções de inserção (alto *regret value*), enquanto a Best Insertion seleciona globalmente o par (job, posição) de menor custo incremental.

Parâmetros de ruído na inserção [JSPRIT-SRC: Jsprit.java]:

- `INSERTION_NOISE_LEVEL = 0.15` — nível de perturbação adicionado ao custo de inserção
- `INSERTION_NOISE_PROB = 0.2` — probabilidade de aplicar ruído em cada avaliação

4. Tratamento de Restrições e Função Objetivo

4.1 Restrições do Modelo

4.1.1 Capacidade do Veículo

A capacidade é modelada através da dimensão 0 (`addCapacityDimension(0, capacity)`) no tipo do veículo, com valor padrão de **100 unidades** (configurável via `globalParameters.vehicleCapacity`) [CÓDIGO: `VrpService.java:93-100`]. O Jsprit verifica automaticamente a restrição de capacidade em cada ponto da rota:

- **Delivery:** o veículo sai do depósito carregado e descarrega no cliente (carga diminui).
- **Pickup:** o veículo coleta no cliente (carga aumenta).
- Em cada ponto: $(\text{entregas restantes} + \text{coletas já realizadas}) \leq \text{capacidade}$.

4.1.2 Simultaneidade de Coleta e Entrega (VRPSPD)

A coleta e entrega simultânea é modelada através de **jobs separados** de tipos **Delivery** e **Pickup** para cada cliente [CÓDIGO: `VrpService.java:184-201`]. Cada cliente que possui tanto demanda de entrega quanto de coleta gera dois jobs distintos (ex: `d_5` para delivery e `p_5` para pickup no cliente 5). A constraint **SameVehicleConstraint** (seção 4.1.3) garante que ambos sejam atendidos pelo mesmo veículo.

4.1.3 Exclusividade do Cliente Compartilhado e Same-Vehicle Constraint

A restrição de que jobs de delivery e pickup do mesmo cliente devem ser atendidos pelo mesmo veículo é implementada via a classe **SameVehicleConstraint**, que implementa **HardRouteConstraint** do Jsprit [CÓDIGO: `SameVehicleConstraint.java:29`]:

- Um mapa `relatedJobs` associa cada par delivery/pickup (ex: `d_5 <-> p_5`) [CÓDIGO: `VrpService.java:204-207`].
- No método `fulfilled(JobInsertionContext)`, ao tentar inserir um job em uma rota, o constraint verifica se o job relacionado já está atribuído a algum veículo. Se estiver, a inserção só é permitida no **mesmo veículo** [CÓDIGO: `SameVehicleConstraint.java:44-61`].
- Um `JobAssignmentUpdater` (implementando `StateUpdater` e `ActivityVisitor`) mantém um mapa `jobToVehicleMap` (`ConcurrentHashMap`) atualizado a cada modificação de rota, rastreando em qual veículo cada job está atribuído [CÓDIGO: `SameVehicleConstraint.java:85-114`].

4.1.4 Restrição de Skills (Modo C8)

No modo C8, a atribuição de clientes a carriers é controlada pelo mecanismo de **skills** do Jsprit [CÓDIGO: `VrpService.java:113-114,189`]:

- Veículos recebem uma skill correspondente ao seu carrier ("`1`" ou "`2`").
- Clientes exclusivos de um carrier recebem `addRequiredSkill(carrierId)`, forçando atendimento apenas pelo veículo correspondente.
- **Clientes compartilhados** não recebem skill restritiva, permitindo atribuição a qualquer veículo — a decisão de alocação é delegada ao solver.

4.1.5 Alocação de Clientes Compartilhados (Modo Custom)

No modo Custom, até **8 configurações de alocação** são geradas para clientes compartilhados e testadas via multi-start externo [CÓDIGO: `VrpService.java:554-639`]:

#	Estratégia	Descrição
1	Todos ao carrier 1	Todas as demandas combinadas atribuídas ao carrier 1
2	Todos ao carrier 2	Todas as demandas combinadas atribuídas ao carrier 2
3	Separados	Cada carrier atende sua própria demanda (2 visitas ao cliente)
4	Proximidade	Cliente atribuído ao carrier cujo depósito é mais próximo
5	Proximidade inversa	Cliente atribuído ao carrier cujo depósito é mais distante
6-8	Combinatórias/Aleatórias	Se ≤3 clientes compartilhados: enumeração binária. Se >3: alocações aleatórias com semente fixa (42)

4.2 Função Objetivo

A função objetivo minimiza o **custo total de transporte** [CÓDIGO: `VrpService.java:96-106`], calculado como:

$$Z = \sum (\text{distância percorrida} \times \text{custo por distância})$$

Onde:

- **Custo por distância** (`costPerDistance`) = **1.0** para todos os tipos de veículo [CÓDIGO: `VrpService.java:98`].
- **Custo fixo do veículo** (`fixedCost`) = **0.0** (sem penalidade por uso de veículo adicional) [CÓDIGO: `VrpService.java:99`].
- **Distância de transporte = Tempo de transporte** (ambos definidos com o mesmo valor da matriz de custos) [CÓDIGO: `VrpService.java:78-79`].
- A matriz de custos é **assimétrica** (`Builder.newInstance(true)`) [CÓDIGO: `VrpService.java:73`].

O custo da solução (`solution.getCost()`) retornado pelo Jsprit representa a soma dos custos de todas as rotas, incluindo penalizações internas para jobs não atribuídos (*unassigned jobs penalty*) [JSPRIT-SRC: `VehicleRoutingProblemSolution`].

5. Critérios de Parada e Parâmetros

5.1 Critério de Parada

O critério de parada é exclusivamente baseado no **número máximo de iterações** do algoritmo Ruin and Recreate (`algorithm.setMaxIterations()`) [CÓDIGO: `VrpService.java:413,449`]. O padrão do Jsprit é 2.000 iterações [JSPRIT-SRC: `Jsprit.java`], mas este projeto sobrescreve esse valor conforme o modo. Não há critério de parada por tempo ou por convergência configurado explicitamente.

5.2 Parâmetros por Modo de Resolução

Parâmetro	Modo C8 (colaborativo)	Modo C8 (sem colaboração)	Modo Custom (colaborativo)	Modo Custom (sem colaboração)
Iterações por execução	1.000	1.000	1.000	2.000
Número de multi-starts	10	10 (por carrier)	2 (por alocação) × até 8 alocações	5 (por carrier)
Threads por execução	4	4	4	8
Vehicle Switch	true	true	false	true
Fast Regret	false	false	false	false
Threshold Alpha	padrão Jsprit (0.15)	padrão Jsprit (0.15)	padrão Jsprit (0.15)	0.1

Parâmetro	Modo C8 (colaborativo)	Modo C8 (sem colaboração)	Modo Custom (colaborativo)	Modo Custom (sem colaboração)
Threshold Ini	padrão Jsprit (0.03)	padrão Jsprit (0.03)	padrão Jsprit (0.03)	0.05
Semente aleatória	$seed \times 1000 + 42$	$seed \times 1000 + 42$	$seed \times 1000 + 42$	$seed \times 1000 + 42$
Fleet Size	INFINITE	INFINITE	INFINITE	INFINITE

[CÓDIGO: `VrpService.java:404-411 (solveWithMultiStart)`, `441-449 (solveWithConstraintMultiStart)`]

5.3 Descrição dos Parâmetros

- **Iterações (maxIterations):** Número de ciclos completos de Ruin and Recreate executados por cada instância do algoritmo [CÓDIGO].
- **Multi-starts:** Número de execuções independentes do algoritmo com sementes diferentes. A melhor solução entre todas as execuções é retida [CÓDIGO].
- **Threads:** Número de threads utilizadas para paralelização interna do Jsprit (avaliação simultânea de inserções) [CÓDIGO].
- **Vehicle Switch:** Quando `true`, permite que o algoritmo transfira jobs entre veículos durante a fase de inserção. Quando `false`, jobs só podem ser reinseridos no veículo de onde foram removidos [JSPRIT-SRC: `Jsprit.java`]. [CÓDIGO: `VrpService.java:406,443`].
- **Fast Regret:** Quando `true`, ativa a versão otimizada da Regret Insertion com filtragem espacial. Quando `false` (como neste projeto), a Regret Insertion padrão é usada [JSPRIT-SRC: `Jsprit.java`; JSPRIT-DOC: `GraphHopper Forum`]. [CÓDIGO: `VrpService.java:407,444`].
- **Threshold Alpha (α):** Controla a velocidade de decaimento exponencial do limiar de aceitação. Valor menor $\rightarrow$ decaimento mais rápido $\rightarrow$ busca mais gulosa. Padrão Jsprit: 0.15. Modo Custom sem colaboração: 0.1 [JSPRIT-SRC: `SchrimpfAcceptance.java`]. [CÓDIGO: `VrpService.java:408`].
- **Threshold Ini (θ_0):** Limiar inicial de aceitação, como fração do custo da melhor solução. Uma solução vizinha é aceita se $custo_novo < custo_pior_aceito + threshold(i)$. Padrão Jsprit: 0.03 (3%). Modo Custom sem colaboração: 0.05 (5%) [JSPRIT-SRC: `SchrimpfAcceptance.java`]. [CÓDIGO: `VrpService.java:409`].
- **Semente aleatória:** Fórmula determinística $seed \times 1000 + 42$ garante reprodutibilidade entre execuções [CÓDIGO: `VrpService.java:410,446`].
- **Fleet Size INFINITE:** O Jsprit pode criar múltiplas instâncias de cada tipo de veículo conforme necessário [CÓDIGO: `VrpService.java:247,735,982`].

5.4 Parâmetros Padrão do Jsprit (não sobrescritos no código)

Os seguintes parâmetros usam valores padrão do Jsprit e **não são configurados explicitamente** no código deste projeto [JSPRIT-SRC: `Jsprit.java`]:

Parâmetro	Valor Padrão	Descrição
CONSTRUCTION	REGRET_INSERTION	Heurística de construção da solução inicial

Parâmetro	Valor Padrão	Descrição
REGRET_K	2	Nível de regret (compara 2ª melhor vs melhor inserção)
REGRET_K_STRATEGY	"sum"	Estratégia de agregação do regret
INSERTION_NOISE_LEVEL	0.15	Nível de ruído na avaliação de inserção
INSERTION_NOISE_PROB	0.2	Probabilidade de aplicar ruído
RUIN_WORST_NOISE_LEVEL	0.15	Nível de ruído no Worst Ruin
RUIN_WORST_NOISE_PROB	0.2	Probabilidade de ruído no Worst Ruin

5.5 Esforço Computacional Total

Modo	Cálculo	Total de iterações Ruin & Recreate
C8 colaborativo	10 starts × 1.000 iter	10.000
C8 sem colaboração	2 carriers × 10 starts × 1.000 iter	20.000
Custom colaborativo	até 8 alocações × 2 starts × 1.000 iter	até 16.000
Custom sem colaboração	2 carriers × 5 starts × 2.000 iter	20.000

[CÓDIGO: `VrpService.java:264,753,985,413,449`]

6. Referências e Fontes

6.1 Referências Acadêmicas

1. **Schrimpf, G., Schneider, J., Stamm-Wilbrandt, H., & Dueck, G.** (2000). Record Breaking Optimization Results Using the Ruin and Recreate Principle. *Journal of Computational Physics*, 159(2), 139–171. DOI: [10.1006/jcph.1999.6413](https://doi.org/10.1006/jcph.1999.6413)
2. **Pisinger, D., & Ropke, S.** (2007). A general heuristic for vehicle routing problems. *Computers & Operations Research*, 34(8), 2403–2435. DOI: [10.1016/j.cor.2005.09.012](https://doi.org/10.1016/j.cor.2005.09.012)
3. **Ropke, S., & Pisinger, D.** (2006). An Adaptive Large Neighborhood Search Heuristic for the Pickup and Delivery Problem with Time Windows. *Transportation Science*, 40(4), 455–472. DOI: [10.1287/trsc.1050.0135](https://doi.org/10.1287/trsc.1050.0135)

6.2 Software

4. **GraphHopper.** jsprit: A java based, open source toolkit for solving rich vehicle routing problems. GitHub. Disponível em: <https://github.com/graphhopper/jsprit>

6.3 Fontes Consultadas do Código-Fonte do Jsprit

As informações marcadas como `[JSPRIT-SRC]` foram verificadas nas seguintes classes do repositório `graphhopper/jsprit` no GitHub:

- `Jspnit.java` — Configuração padrão do algoritmo, operadores, pesos e parâmetros
- `SchrimpfAcceptance.java` — Critério de aceitação por threshold
- `docs/Meta-Heuristic.md` — Documentação oficial da meta-heurística

6.4 Fontes Consultadas do Código-Fonte deste Projeto

As informações marcadas como `[CÓDIGO]` foram verificadas diretamente nos seguintes arquivos:

- `src/main/java/com/pibic/vrp/service/VrpService.java` — Lógica principal do solver
- `src/main/java/com/pibic/vrp/constraint/SameVehicleConstraint.java` — Constraint customizada
- `pom.xml` — Versão da dependência Jspnit